DOSSIER TÉCNICO EXHAUSTIVO (HAND-OFF DOCUMENT)

PROYECTO: FLASHMENTE (PWA & ANDROID WRAPPER)

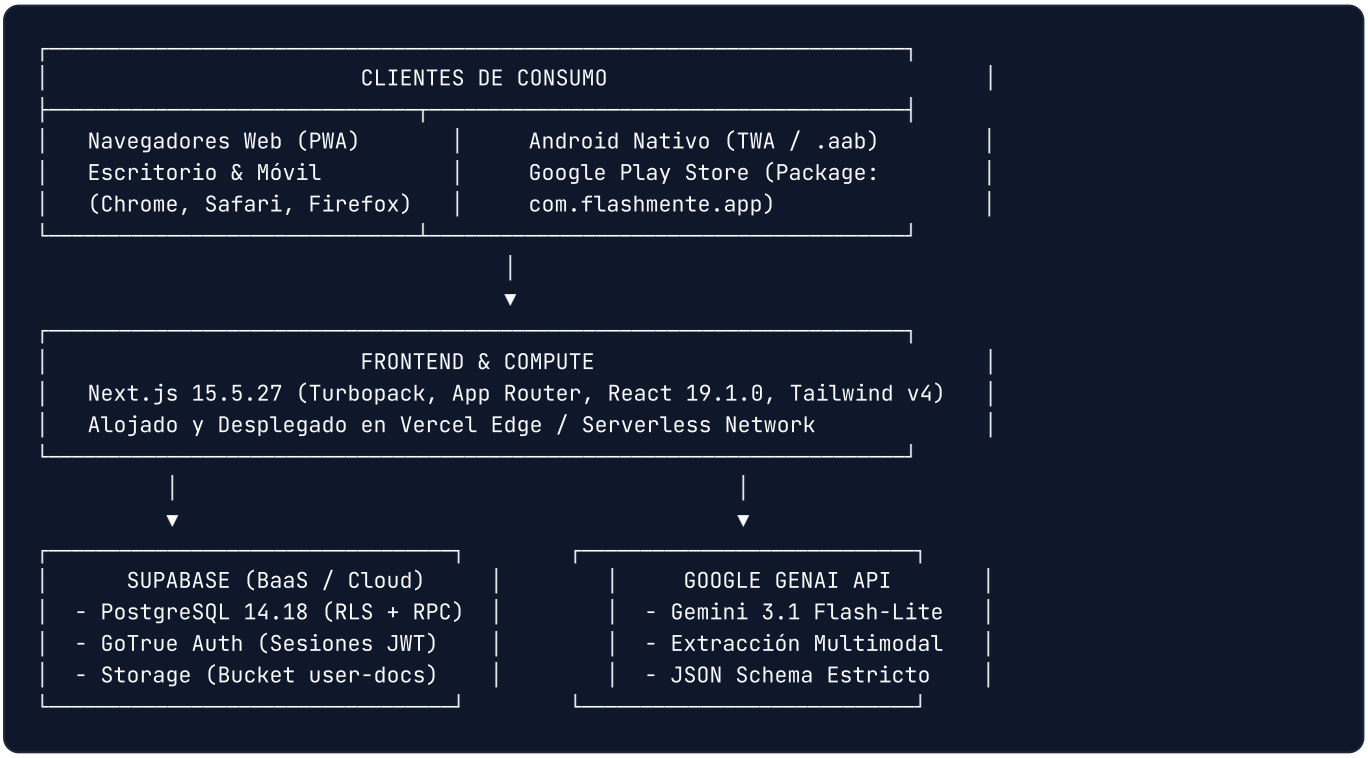
Fecha de emisión: 11 de octubre de 2026
Versión de producción: 1.0.1 (Build / VersionCode: 2)
Dominio de producción: https://flashmente.com / https://www.flashmente.com
Identificador Android (Package ID): `com.flashmente.app`
Repositorio GitHub: `nachomoltol/flashcards-app` (Rama principal: `main`)

---

1. ARQUITECTURA Y STACK TECNOLÓGICO

1.1. Visión General de la Solución

Flashmente es una plataforma de aprendizaje acelerado y memorización a largo plazo basada en repetición espaciada científica (FSRS) y extracción pedagógica multimodal mediante Inteligencia Artificial (Google Gemini). El sistema está diseñado bajo una arquitectura de código único (Single Codebase) estructurada como una Progressive Web App (PWA) de alto rendimiento, encapsulada como aplicación nativa Android mediante Trusted Web Activity (TWA).



1.2. Frontend Framework y Runtime

- **Next.js 15.5.27 con Turbopack:** Utiliza el modelo App Router con soporte para Server Components (RSC) para vistas estáticas y Server Actions para mutaciones backend seguras con acceso a variables de entorno sin exponer credenciales al cliente.
- **React 19.1.0:** Renderizado concurrente, hooks modernos y gestión optimizada de transiciones.
- **Tailwind CSS v4 (`@tailwindcss/postcss`):** Sistema de diseño atómico moderno con directivas simplificadas en `src/app/globals.css`, eliminando sobrecarga de configuración tradicional.
- **TypeScript 5 (Strict Mode):** Tipado estático riguroso en interfaces de base de datos (`src/types/database.ts`), tarjetas de estudio (`src/types/cards.ts`) y lógica de negocio.
- **Gestión de Estado (Zustand 5.0.15):** Stores modulares y desacoplados:
  - `useAuthStore`: Estado de autenticación del usuario, inicialización y cierre de sesión.
  - `useProfileStore`: Perfil, retención deseada FSRS, avatares y preferencias.
  - `useTutorialStore`: Control de apertura y navegación del modal de onboarding interactivo.
  - `useProModalStore`: Estado modal de monetización y planes PRO.
  - `useLanguageStore`: Internacionalización y catálogo de cadenas de texto (ES/EN).

1.3. Arquitectura PWA (Progressive Web App)

La aplicación unifica la experiencia web de escritorio y móvil a través de estándares PWA modernos:

- `public/manifest.json`:
  - `name`: "Flashmente"
  - `short_name`: "Flashmente"
  - `start_url`: "/"
  - `display`: "standalone" (elimina barras de dirección y controles del navegador)
  - `orientation`: "portrait-primary" (fijado a orientación vertical en dispositivos móviles)
  - `theme_color`: "#FFD700" (color dorado de la barra de sistema)
  - `background_color`: "#ffffff"
  - Iconos oficiales: `/icons/android-chrome-192x192.png` y `/icons/android-chrome-512x512.png` con propósito `"any maskable"`.
- **Service Worker (`public/sw.js`):**
  - Versión de caché: `flashmente-pwa-v3`.
  - Estrategia de red: **Network-First con fallback a CacheStorage** para garantizar que los usuarios siempre obtengan la última versión desplegada si tienen conexión, preservando la capacidad de operar sin conexión ante fallos de red.
  - Auto-limpieza: Escucha el evento `activate` para purgar versiones anteriores de caché y reclama clientes activos inmediatamente (`clients.claim()`).
- **Registro Controlado (`src/components/pwa/ServiceWorkerRegister.tsx`):**
  - En entorno de desarrollo (`development`), el componente desregistra cualquier Service Worker activo y vacía la caché local para impedir que el código en caliente (HMR) sea bloqueado.

- En producción, registra `/sw.js`, sondea actualizaciones y envía `postMessage({ type: 'SKIP_WAITING' })` para que los nuevos despliegues tomen el control sin requerir reinicios manuales del navegador.

1.4. Despliegue en Vercel

- **Integración Continua (CI/CD):** Vinculada automáticamente al repositorio GitHub `nachomoltol/flashcards-app`. Cualquier push a la rama `main` desencadena la compilación y despliegue instantáneo en la red global de Vercel.
- **Dominios Asignados:**
  - `flashmente.com` (Redirección permanente 308 configurada en Vercel hacia `www.flashmente.com`).
  - `www.flashmente.com` (Dominio primario canónico con certificado SSL gestionado).
- **Entorno de Variables:** Inyectadas de forma segura en Vercel Environment Variables:
  - `NEXT_PUBLIC_SUPABASE_URL`
  - `NEXT_PUBLIC_SUPABASE_ANON_KEY`
  - `SUPABASE_SERVICE_ROLE_KEY` (si se requiere bypass administrativo)
  - `GEMINI_API_KEY` (clave del modelo multimodal de Google)
  - `STRIPE_SECRET_KEY` / `STRIPE_WEBHOOK_SECRET`

1.5. Backend, Base de Datos y Autenticación (Supabase)

- **Supabase PostgreSQL 14.18 (Project ID: `fapjackhdqtnrpteujum`):**
  - **Tablas Core:**
    - `decks`: Carpetas y mazos de estudio (`id`, `user_id`, `parent_id`, `title`, `description`, `color`, `is_folder`, `share_id`, `is_public`).
    - `cards`: Flashcards asociadas (`id`, `deck_id`, `front`, `back`, `card_format`, `due`, `stability`, `difficulty`, `state`, `reps`, `lapses`, `last_review`).
    - `notes`: Notas maestras con campos semiestructurados (`fields` tipo JSONB).
    - `reviews`: Registro histórico de cada repaso para métricas de retención y curvas de aprendizaje.
    - `profiles`: Datos de usuario, avatares, configuración FSRS personalizada (`request_retention`, `maximum_interval`, `enable_fuzz`).
    - `api_limits`: Control atómico de cuotas de generación de IA (`user_id`, `generations_count`, `last_generation_date`, `bonus_credits`, `tier`: `'free' | 'pro' | 'vip'`).
  - **Seguridad y RLS (Row Level Security):**
    - Todas las tablas principales tienen RLS activado.
    - Funciones RPC con directiva `SECURITY DEFINER` para operaciones atómicas de sistema:
      - `check_api_limit`: Valida si el usuario tiene saldo mensual (10 generaciones gratuitas), créditos adicionales acumulables de anuncios o bypass PRO/VIP sin exponer la lógica al cliente.
      - `increment_api_generation`: Registra el uso mensual o descuenta de `bonus_credits` de anuncios.
      - `add_bonus_credit`: Suma créditos bonificados acumulables tras la visualización exitosa de anuncios de Google AdMob.

- `get_deck_owner`: Permite a los Server Actions verificar la propiedad del mazo.

---

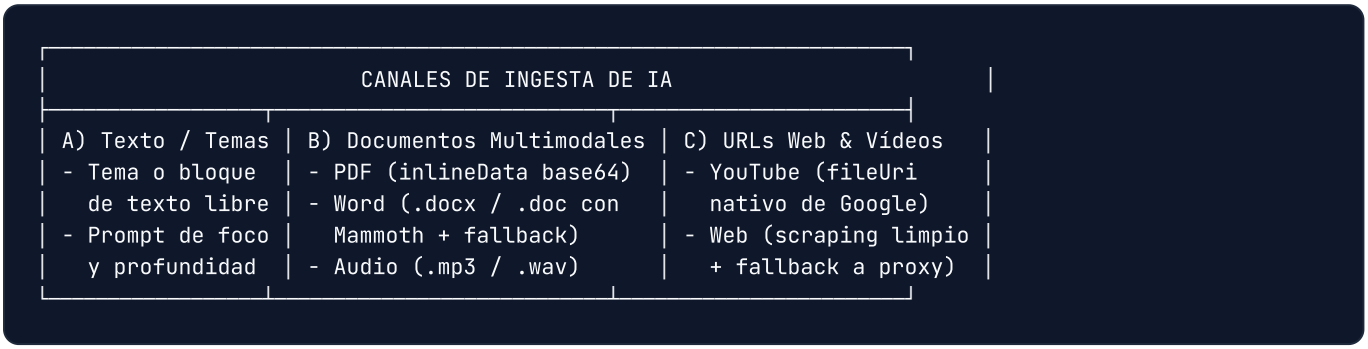
2. INTEGRACIÓN DE INTELIGENCIA ARTIFICIAL (CORE MULTIMODAL)

2.1. Orquestación del SDK y Selección de Modelo

- **SDK Oficial:** `@google/genai` (versión `^2.27.0`) con la clase unificada `GoogleGenAI`.
- **Modelo Utilizado:** `gemini-3.1-flash-lite` (referenciado internamente en `generateCards.ts`). Proporciona una ventana de contexto masiva (hasta 1 millón de tokens), baja latencia de respuesta y coste ultra-sostenible para generaciones ilimitadas.

2.2. Ingesta de Datos Multicanal

La arquitectura en `src/app/actions/generateCards.ts` soporta 3 canales de ingesta:



1. **Documentos PDF y Audio:**
   - Se suben al bucket `user-documents` de Supabase Storage.
   - El Server Action descarga el archivo o recupera una Signed URL y lo transmite a Gemini como `inlineData` estructurado con su `mimeType` correspondiente (`application/pdf`, `audio/mpeg`, etc.).
2. **Archivos de Word (`.docx` / `.doc`):**
   - Para prevenir incompatibilidades con la estructura ZIP de los archivos OpenXML al procesarlos como binarios crudos, el servidor procesa el búfer en memoria mediante la biblioteca `mammoth` (`mammoth.extractRawText`).
   - Cuenta con un mecanismo de fallback para documentos binarios antiguos `.doc` mediante saneamiento por expresiones regulares de caracteres de control ASCII. El texto limpio se inyecta directamente como contexto textual.
3. **Vídeos de YouTube:**
   - **Estrategia Primaria (Nativa de Google):** Se extrae el ID canónico del vídeo y se envía a Gemini utilizando la referencia nativa `fileData: { fileUri: "https://www.youtube.com/watch?v=..." }`. Esta vía se procesa directamente en los servidores de Google, **eliminando al 100% los bloqueos de IP (HTTP 429 / 403) comunes en entornos serverless como Vercel**.
   - **Estrategia Secundaria (Fallback):** En caso de restricción en el vídeo, se activa la extracción de subtítulos mediante `youtube-transcript`.
4. **Artículos y Enlaces Web:**

- Descarga directa con cabecera de navegador estándar (`User-Agent` moderno).
- Función `extractTextFromHtml` que despoja scripts, estilos, cabeceras, footers, navbars y svg.
- Fallback automático mediante el servicio de proxy `allorigins.win` si el sitio de destino bloquea la IP del centro de datos.

2.3. Estructuración y Salida JSON Determinista

Para evitar respuestas divagantes o formatos markdown inestables, la llamada a Gemini exige un esquema estricto de respuesta mediante la API nativa de Google GenAI:

```
const flashcardSchema = {
  type: Type.OBJECT,
  description: 'Objeto con tarjetas de estudio y radar de profundidad de contenido',
  properties: {
    core_exhausted: {
      type: Type.BOOLEAN,
      description: 'true si la teoría troncal ya ha sido extraída; false si aún queda información tron
    },
    cards: {
      type: Type.ARRAY,
      items: {
        type: Type.OBJECT,
        properties: {
          front: { type: Type.STRING },
          back: { type: Type.STRING },
          cardFormat: { type: Type.STRING },
        },
        required: ['front', 'back', 'cardFormat'],
      },
    },
  },
  required: ['core_exhausted', 'cards'],
};
```

Configuración de la llamada:

```
config: {
  systemInstruction: SYSTEM_INSTRUCTION_GROUNDING,
  responseMimeType: 'application/json',
  responseJsonSchema: flashcardSchema,
  maxOutputTokens: 8192,
}
```

2.4. Directivas Pedagógicas y Filtros Core

- **Grounding Estricto (`SYSTEM_INSTRUCTION_GROUNDING`):** Prohíbe de forma terminante el uso de conocimiento externo. Si un dato no figura explícitamente en el material aportado, la IA no puede inventarlo ni completarlo.
- **Radar de Profundidad (`RADAR_DIRECTIVE`):** Si los conceptos troncales ya se han cubierto en lotes anteriores, la IA activa `core_exhausted: true` pero **no se detiene**: continúa profundizando en detalles específicos, excepciones y datos minuciosos hasta alcanzar la cantidad solicitada.

- **Memoria Anti-Duplicados (`buildAntiDuplicateSection`):** Consulta en tiempo real las tarjetas que ya residen en el mazo dentro de Supabase y las inyecta en el prompt con prohibición explícita de repetir conceptos o preguntas análogas.
- **Aleatorización Fisher-Yates:** Las tarjetas tipo test (`multiple_choice`) son post-procesadas en el servidor con el algoritmo Fisher-Yates (`randomizeMultipleChoiceCard`) para garantizar que la respuesta correcta quede distribuida de forma estrictamente equiprobable entre las opciones a, b, c y d.
- **Límite de Seguridad por Lote:** Limitado a un máximo de 30 tarjetas por ejecución para asegurar que la respuesta quepa holgadamente dentro del límite de 8,192 tokens de salida sin truncamiento JSON.

---

3. LÓGICA DE NEGOCIO Y ALGORITMOS CORE (FSRS)

3.1. Modelo Matemático FSRS (Free Spaced Repetition Scheduler)

Flashmente implementa el algoritmo **FSRS v5** (considerado el estándar de oro en ciencias de la memoria, superando ampliamente a SM-2 de Anki clásico), a través del motor oficial `ts-fsrs` (v5.4.2), encapsulado en `src/lib/fsrs/scheduler.ts`.

El algoritmo modela la memoria humana utilizando tres variables fundamentales:

1. **Estabilidad (S):** Tiempo (en días) que tarda la probabilidad de recuerdo en descender del 100% al 90%.
2. **Dificultad (D):** Resistencia intrínseca del concepto a ser fijado en la memoria (escala de 1 a 10).
3. **Retentividad (R):** Probabilidad instantánea de recuperar el concepto en el momento del repaso: $$R(t) = \left(1 + \text{factor} \cdot \frac{t}{S}\right)^{\text{potencia}}$$

3.2. Calificaciones y Curva de Pesos Personalizada

El usuario califica su recuerdo mediante 4 opciones:

- `1 = Again` **(Repetir):** Fallo total. La tarjeta entra en reaprendizaje. El intervalo se reprograma a 10 minutos (`learning_steps: ['10m']`).
- `2 = Hard` **(Difícil):** Recuerdo trabajoso. Para tarjetas nuevas, se fuerza un intervalo controlado de **3 horas** para afianzar el concepto en el mismo día.
- `3 = Good` **(Correcto):** Recuerdo adecuado. Peso optimizado a **1.2 días** en el primer ciclo.
- `4 = Easy` **(Fácil):** Dominio completo. Peso optimizado a **3.0 días** iniciales, expandiendo exponencialmente los intervalos futuros.

```
export const initialFSRSWeights: number[] = [...default_w];
initialFSRSWeights[2] = 1.2; // Good inicial: ~1.2 días
initialFSRSWeights[3] = 3.0; // Easy inicial: ~3.0 días

export const defaultFSRSParams = generatorParameters({
  enable_fuzz: true,           // Evita avalanchas de repaso acumuladas en el mismo día
  learning_steps: ['10m'],    // Intervalo de reintento ante fallos
  relearning_steps: ['10m'],
  w: initialFSRSWeights,
});
```

3.3. Ciclo de Vida de los Estados de una Tarjeta

Las tarjetas transitan por 4 estados definidos en el enum `State`:

- `State.New (0)`: Tarjeta creada que nunca ha sido estudiada.
- `State.Learning (1)`: Tarjeta en fase de aprendizaje inicial.
- `State.Review (2)`: Tarjeta consolidada en repaso espaciado estándar.
- `State.Relearning (3)`: Tarjeta olvidada tras haber estado en fase de repaso.

3.4. Sincronización y Persistencia en Supabase

Cada vez que el usuario responde en el modo de estudio (`src/app/study/[deckId]/page.tsx`), la función `scheduleReviewWithLog` calcula el nuevo estado y ejecuta dos operaciones en Supabase:

1. **Actualización de la fila en `cards`:**
   - `due`: Nueva fecha exacta de vencimiento (ISO 8601).
   - `stability`: Nuevo valor de estabilidad (S).
   - `difficulty`: Nuevo valor de dificultad (D).
   - `state`: Nuevo estado numérico.
   - `reps`: Contador de repasos totales incrementado.
   - `lapses`: Contador de fallos (se incrementa si la calificación fue `Again`).
   - `last_review`: Marca de tiempo del momento del estudio.
2. **Inserción de auditoría en `reviews`:**
   - Permite graficar el historial de aciertos, la retención proyectada y la racha activa del estudiante en `/stats`.

---

4. INFRAESTRUCTURA DE DESPLIEGUE ANDROID (TWA / WRAPPER)

4.1. Tecnología del Envoltorio Móvil

Para publicar la PWA en Google Play Store sin la sobrecarga ni el mantenimiento de múltiples bases de código nativas (como Kotlin o Java manual), se utiliza la tecnología **Trusted Web Activity (TWA)** respaldada por Google mediante las bibliotecas de `Android Browser Helper`.

4.2. Identidad y Metadatos de la Aplicación

- **Nombre de la App:** Flashmente
- **Application ID / Package Name:** `com.flashmente.app`
- **Versión de Producción:**
  - `versionCode`: `2`
  - `versionName`: `1.0.1`
- **Nivel de API Android (Target SDK):**

- `minSdkVersion`: `24` (Android 7.0 Nougat o superior, cubriendo >96% del parque global de dispositivos).
- `compileSdkVersion` / `targetSdkVersion`: `34` / `36` (Cumplimiento riguroso de las políticas más recientes de Google Play Console).

4.3. Claves Criptográficas de Firma (Keystore)

El binario está firmado con el certificado oficial del proyecto:

- **Ubicación en el entorno de compilación:** `C:\Users\Nacho\OneDrive\Desktop\PROYECTO APP DE FLASCARDS CLAVES\Flashmente - Google Play package 1.0.1\signing.keystore`
- **Alias de la clave:** `flashmente-key`
- **Contraseña del Keystore:** `DbIbvnfxT5M9`
- **Contraseña de la Clave:** `DbIbvnfxT5M9`
- **Huella Digital SHA-256 (Upload Key):** `DB:A6:51:DF:7F:C0:20:FF:64:21:9B:8A:5D:A2:4E:FB:31:94:10:F3:94:D6:48:E2:C1:2D:00:AA:0B:2C:50:62`
- **Huella Digital SHA-256 (Play App Signing de Google):** `34:56:38:55:A2:B1:C0:C8:F8:4E:D1:18:89:CF:25:70:42:FE:E3:02:26:D0:1E:02:25:E9:AC:6E:5F:6F:49:D1`

4.4. Vinculación Digital (Digital Asset Links)

Para que el sistema operativo Android verifique que la aplicación nativa es propietaria del dominio web y **elimine automáticamente la barra de navegación del navegador (URL bar)**, se publica el archivo de verificación en `public/.well-known/assetlinks.json`:

```
[
  {
    "relation": ["delegate_permission/common.handle_all_urls"],
    "target": {
      "namespace": "android_app",
      "package_name": "com.flashmente.app",
      "sha256_cert_fingerprints": [
        "34:56:38:55:A2:B1:C0:C8:F8:4E:D1:18:89:CF:25:70:42:FE:E3:02:26:D0:1E:02:25:E9:AC:6E:5F:6F:49:
        "DB:A6:51:DF:7F:C0:20:FF:64:21:9B:8A:5D:A2:4E:FB:31:94:10:F3:94:D6:48:E2:C1:2D:00:AA:0B:2C:50:
      ]
    }
  }
]
```

(Nota: Incluye ambas huellas digitales para garantizar validación tanto en compilaciones locales de depuración como en la versión firmada y distribuida por la tienda de Google Play).

4.5. Pipeline de Compilación Automatizado (.aab)

La compilación del archivo **Android App Bundle (.aab)** se ejecuta mediante la API del servicio Cloud de PWABuilder (respaldado por Azure y contenedores con el Android SDK oficial de Google y Bubblewrap):

1. El script lee el archivo `signing.keystore`, lo codifica en Base64 con esquema de datos URI (`data:application/octet-stream; base64,...`).
2. Realiza un `POST` contra `https://pwabuilder-cloudapk.azurewebsites.net/enqueuePackageJob` con la configuración completa (`host: flashmente.com`, `packageId: com.flashmente.app`, `versionCode: 2`, `appVersion: 1.0.1`).
3. El servicio devuelve un ticket con `id` y `accessToken`.
4. Se sondea `/getPackageJob` hasta recibir el estado `Completed`.
5. Se descarga el archivo ZIP resultante desde `/downloadPackageZip`.
6. El archivo resultante Flashmente.aab (~5.66 MB) se extrae y se coloca en el Escritorio para su subida inmediata a la consola de Google Play.

---

5. CONFIGURACIONES DE RED, SEO Y COMPLIANCE

5.1. Configuración de `next.config.ts`

El archivo de configuración principal de Next.js incluye reescrituras de rutas y cabeceras de seguridad y permisos:

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  async rewrites() {
    return [
      {
        source: '/privacy',
        destination: '/privacidad',
      },
    ];
  },
  async headers() {
    return [
      {
        source: '/(privacidad|privacy)',
        headers: [
          {
            key: 'X-Robots-Tag',
            value: 'all, index, follow',
          },
          {
            key: 'Access-Control-Allow-Origin',
            value: '*',
          },
          {
            key: 'Access-Control-Allow-Methods',
            value: 'GET, HEAD, OPTIONS',
          },
          {
            key: 'Cache-Control',
            value: 'public, max-age=3600, s-maxage=86400, stale-while-revalidate',
          },
        ],
      },
      {
        source: '/robots.txt',
        headers: [
          {
            key: 'Content-Type',
            value: 'text/plain; charset=utf-8',
          },
          {
            key: 'Cache-Control',
            value: 'public, max-age=86400',
          },
        ],
      },
      {
        source: '/.well-known/assetlinks.json',
        headers: [
          {
            key: 'Content-Type',
            value: 'application/json',
          },
          {
            key: 'Access-Control-Allow-Origin',
            value: '*',
          },
          {
            key: 'Cache-Control',
```

```
            value: 'public, max-age=86400',
          },
        ],
      },
    ];
  },
};

export default nextConfig;
```

5.2. Control de Robots y Sitemaps

- `public/robots.txt`:
  Permite de forma explícita el rastreo a todos los motores de búsqueda y bots de revisión de Google (`Googlebot`, `Google-Play-Review-Bot`, etc.):

  ```
  User-agent: *
  Allow: /
  Allow: /privacidad
  Allow: /privacy
  Allow: /.well-known/assetlinks.json

  Sitemap: https://flashmente.com/sitemap.xml
  ```

- `public/sitemap.xml`:
  Estructura XML estándar que lista la raíz `/`, `/privacidad` y `/privacy` con sus frecuencias de actualización y prioridades.

5.3. Página de Política de Privacidad y Cumplimiento con Google Play

- **Ubicación del Código:** `src/app/privacidad/page.tsx` (con alias en `src/app/privacy/page.tsx`).
- **Naturaleza del Componente:** Es un **Server Component puro** que no depende de `'use client'`, no carga JavaScript obligatorio en el cliente y no requiere ninguna cookie ni autenticación en Supabase. Se genera de forma 100% estática (`○ (Static)`) durante el build.
- **Resolución del Error "Permission Denied" en Google Play:**
  - Anteriormente, la ausencia de un `robots.txt` provocaba que la petición del bot devolviera una página 404 con `<meta name="robots" content="noindex">`.
  - La redirección 308 de `flashmente.com` a `www.flashmente.com` confundía a ciertos evaluadores automáticos.
  - Al inyectar cabeceras CORS universales (`*`), `X-Robots-Tag: all, index, follow` y un `robots.txt` estático, tanto `https://flashmente.com/privacidad` como `https://www.flashmente.com/privacidad` devuelven respuesta limpia e inmediata.
- **Cláusulas Legales Obligatorias Integradas:**
  1. Tipología exacta de datos recopilados (correo electrónico, nombre opcional, ID interno).
  2. Finalidad exclusiva (autenticación, gestión de cuenta, sincronización cloud de mazos).
  3. No comercialización ni cesión a terceros.
  4. Lista de subencargados del tratamiento (Supabase para base de datos/auth, Vercel para hosting, Google Gemini para IA).

5. Cifrado en tránsito completo mediante HTTPS/SSL.
6. **Mecanismo explícito de supresión de cuenta y borrado de datos:** Instrucción clara para solicitar la eliminación permanente escribiendo a `nachomoltol@gmail.com` con plazo de resolución inferior a 30 días.

---

6. ESTRUCTURA DE UX/UI Y ONBOARDING

6.1. Refactorización y Ergonomía Móvil

Para garantizar una experiencia fluida con una sola mano en pantallas de teléfonos móviles:

1. **Acceso directo en '+ Nuevo':** Se suprimieron los menús intermedios tipo dropdown o bottom-sheet. Al pulsar el botón principal `+ Nuevo`, se despliega de inmediato el modal unificado con pestañas intercambiables para crear Carpeta o Mazo de Estudio.
2. **Botón de Gestión Rápida (⚙️):** Cada tarjeta de mazo o carpeta en la cuadrícula de inicio incluye un botón de engranaje visible junto al botón de acción principal (`Estudiar` o `Abrir`). Esto permite renombrar, modificar el color temático o eliminar el recurso sin depender de pulsaciones largas o menús contextuales difíciles de pulsar.
3. **Optimización de Espacio:** En resoluciones pequeñas (`< 768px`), se ocultan los breadcrumbs y las tarjetas grandes de estadísticas globales en la raíz, dejando el temario y los mazos en una cuadrícula compacta y limpia de 2 columnas.

6.2. Arquitectura del Onboarding Interactivo (5 Pasos)

El tutorial de bienvenida se gestiona mediante el componente `src/components/onboarding/WelcomeTutorial.tsx` y el store `useTutorialStore`:

- **Persistencia Local:** Al completarse o descartarse, almacena la bandera `flashcards_welcome_tutorial_seen = 'true'` en `localStorage`.
- **Enfoque 100% Práctico:** Orientado a la usabilidad real en lugar de mensajes promocionales:
  - **Paso 1 - Crea tu primera Carpeta:** Explica cómo organizarse usando el botón principal `+ Nuevo`.
  - **Paso 2 - Genera preguntas al instante:** Guía sobre cómo subir PDFs, Word, audio o enlaces de YouTube a través de la IA.
  - **Paso 3 - El arte del repaso:** Instruye sobre la honestidad al valorar el recuerdo y la correspondencia de los botones 1, 2, 3 o 4 con el algoritmo de repetición espaciada.
  - **Paso 4 - Edita y organiza a tu gusto:** Muestra el uso del icono de engranaje (⚙️) para cambiar nombres, colores o borrar elementos.
  - **Paso 5 - Tu turno. ¡A jugar!:** Invita a abrir el mazo de prueba para realizar la primera sesión práctica interactiva.

6.3. Mazo Predeterminado 'Guía Rápida'

El archivo `src/lib/onboarding/seedGuideDeck.ts` implementa la función `ensureQuickGuideDeck(userId)`, la cual es invocada de forma idempotente tras el registro o inicio de sesión:

- Crea el mazo `"Guía Rápida"` con color distintivo índigo.
- Siembra 5 flashcards interactivas con preguntas sobre el uso práctico de la plataforma:
  1. *¿Cuál es el botón más importante de la pantalla de inicio para empezar a organizar tu temario?* ➔ El botón `+ Nuevo`.
  2. *Tienes un PDF de 20 páginas y no quieres escribir las preguntas a mano. ¿Qué haces?* ➔ Usar `Generar preguntas con IA` en bloques de hasta 30 tarjetas.
  3. *Acabas de voltear esta tarjeta. Si te ha parecido facilísima y te la sabes de memoria, ¿qué botón debes pulsar ahora mismo?* ➔ El botón azul `Easy` (o la tecla 4).
  4. *Has creado un mazo pero quieres cambiarle el color o borrarlo por completo. ¿Dónde tocas?* ➔ En el icono del engranaje (⚙️).
  5. *Te has quedado en blanco con una pregunta. Pulsas 'Again' (botón rojo o tecla 1). ¿Qué ocurrirá con esa tarjeta?* ➔ Volverá a aparecer al final de la sesión de estudio.

---

7. COMANDOS CLAVE PARA DESARROLLO Y MANTENIMIENTO

```
# 1. Instalación de dependencias
npm install

# 2. Ejecución del entorno de desarrollo local (Turbopack)
npm run dev

# 3. Verificación de compilación de producción y tipado TypeScript
npm run build

# 4. Despliegue manual a producción (vía Git)
git add .
git commit -m "feat/fix: descripción del cambio"
git push origin main
# (Vercel detecta el push e inicia la compilación automáticamente)

# 5. Generación de un nuevo bundle Android .aab (si se incrementa la versión)
# Modificar version en scratch/generate_aab.mjs y ejecutar:
node scratch/generate_aab.mjs
```

---

8. RESUMEN DE UBICACIONES Y ARCHIVOS CLAVE

Componente	Ruta en el Proyecto
Página de Privacidad	`src/app/privacidad/page.tsx`
Alias de Privacidad	`src/app/privacy/page.tsx`
Cabeceras y Rewrites	`next.config.ts`
Robots.txt	`public/robots.txt`
Sitemap.xml	`public/sitemap.xml`
Digital Asset Links (TWA)	`public/.well-known/assetlinks.json`
Manifest PWA	`public/manifest.json`
Service Worker	`public/sw.js`
Motor FSRS	`src/lib/fsrs/scheduler.ts`
Acciones de IA (Gemini)	`src/app/actions/generateCards.ts`
Tutorial de Onboarding	`src/components/onboarding/WelcomeTutorial.tsx`
Mazo Demo Seed	`src/lib/onboarding/seedGuideDeck.ts`
Binario Android (.aab)	`Flashmente-1.0.1.aab`
Keystore de Firma	`Flashmente - Google Play package 1.0.1\signing.keystore`